

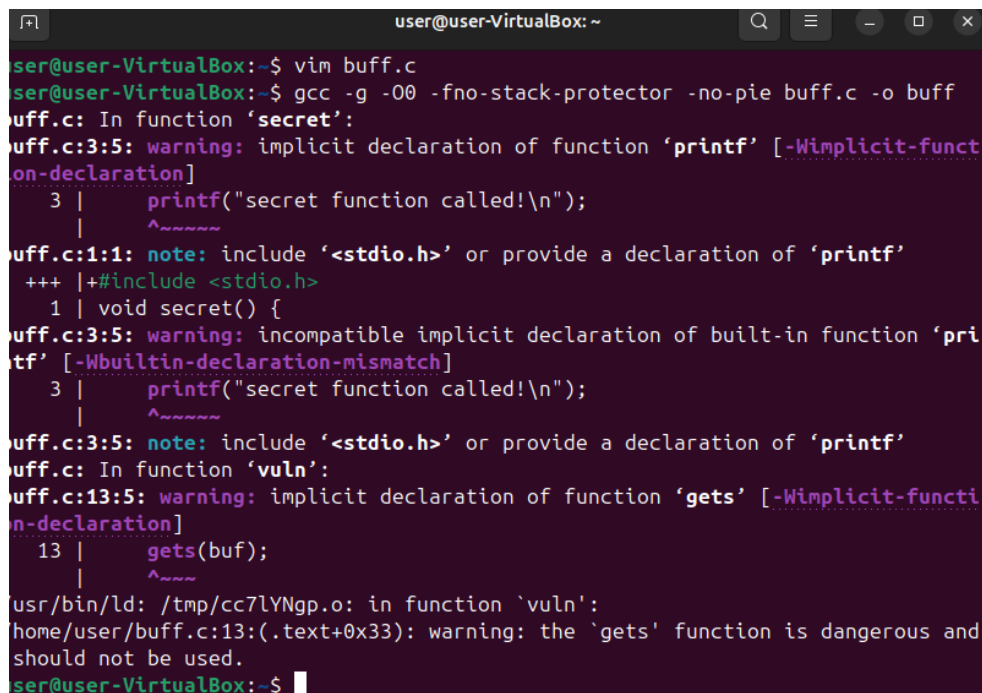
Beginner-C Bootcamp 9주차 과제

20215789 산업보안학과 김문정

<버퍼 오버플로우 익스플로잇>

Challenge 1. ret address 덮어서 실행 흐름 바꾸기

우선 gcc -g -O0 -fno-stack-protector -no-pie 가 어떤 의미인지 몰라서 인공지능과 함께 학습하였더니 최적화를 끄고, canary를 제거하고, 함수 주소를 고정하는 코드라는 것을 알 수 있었다. 이에 저 코드를 이용하여 컴파일을 수행하고 gdb에서 실행시켰다.



```
user@user-VirtualBox: ~  
ser@user-VirtualBox:~$ vim buff.c  
ser@user-VirtualBox:~$ gcc -g -O0 -fno-stack-protector -no-pie buff.c -o buff  
buff.c: In function 'secret':  
buff.c:3:5: warning: implicit declaration of function 'printf' [-Wimplicit-function-declaration]  
    3 |     printf("secret function called!\n");  
      |     ^~~~~  
buff.c:1:1: note: include '<stdio.h>' or provide a declaration of 'printf'  
+++ |+#include <stdio.h>  
    1 | void secret() {  
buff.c:3:5: warning: incompatible implicit declaration of built-in function 'printf' [-Wbuiltin-declaration-mismatch]  
    3 |     printf("secret function called!\n");  
      |     ^~~~~  
buff.c:3:5: note: include '<stdio.h>' or provide a declaration of 'printf'  
buff.c: In function 'vuln':  
buff.c:13:5: warning: implicit declaration of function 'gets' [-Wimplicit-function-declaration]  
   13 |     gets(buf);  
      |     ^~~~~  
usr/bin/ld: /tmp/cc7lYNgp.o: in function 'vuln':  
home/user/buff.c:13:(.text+0x33): warning: the 'gets' function is dangerous and should not be used.  
ser@user-VirtualBox:~$
```

우선 컴파일을 수행하니 마지막에 gets 함수가 위험하다는 경고를 띄워주는 것을 확인할 수 있었다. 그 이유는 5주차에도 학습하였듯, buf의 공간이 16바이트만 할당되었음에도, gets로 사용자의 입력을 받도록 하게 되면 gets는 사용자의 입력에 대해 16바이트를 초과하는지 여부를 검사하지 않기 때문에 초과한 입력에 대해 ret add를 덮는 경우가 발생하게 된다. (버퍼 오버플로우 발생 가능)

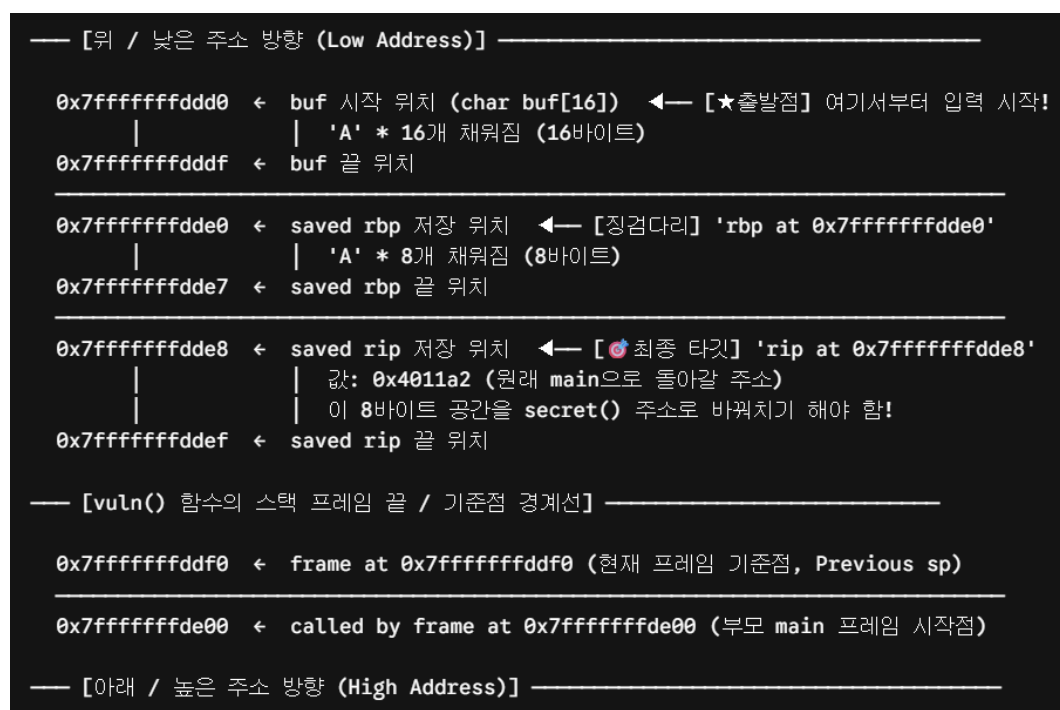
우선 BOF exploit전에 레지스터들과 주소위치에 대해 짚고 넘어가자면 대부분의 컴퓨터에서 주소는 위쪽이 낮은주소(0x00000), 아래쪽이 높은주소(0xFFFFF)를 말한다.

또 rax는 임시 저장 레지스터, rbp는 스택의 바닥 주소, rsp는 스택의 최상단 주소, rip는 다음 실행할 명령어의 주소를 담아두는 레지스터를 말한다.

gdb에서 info frame명령어를 입력해보면

```
(gdb) info frame
Stack level 0, frame at 0x7fffffffddfd0:
 rip = 0x40117c in vuln (buff.c:13); saved rip = 0x4011a2
 called by frame at 0x7fffffffde00
 source language c.
 Arglist at 0x7fffffffdde0, args:
 Locals at 0x7fffffffdde0, Previous frame's sp is 0x7fffffffddfd0
 Saved registers:
  rbp at 0x7fffffffdde0, rip at 0x7fffffffdde8
(gdb)
```

```
(gdb) p &buf
$1 = (char (*)[16]) 0x7fffffffddd0
(gdb) p &secret
$2 = (void (*)()) 0x401156 <secret>
```



위 그림을 바탕으로 설명해보자면 p &buf로 buf의 주소를 출력해보면 0x7fffffffddd0임을 알 수 있고, buf는 16바이트이므로 끝은 0x7fffffffdddf인 것을 알 수 있다. buf가 끝난 후 바로 다음 주소에 스택의 끝부분을 가리키는 레지스터인 rbp가 있는 것을 알 수 있고, 주소는 0x7fffffffdde0이 된다. 이때 rbp가 의미하는 바는 컴퓨터가 vuln() 함수를 끝내고 main() 함수로 돌아갈 때 필요한 원래 main함수의 rbp가 저장되어 있는 공간이다. 또한 왜 rbp가 8바이트 공간인지 설명하자면 대부분 컴퓨터의 실행환경이 64비트여서 값을 저장하는 기본단위인 word가 8바이트이고, 레지스터의 이름에서도 알 수 있다.

rbp (Register Base Pointer): 64비트(8바이트) 크기의 레지스터

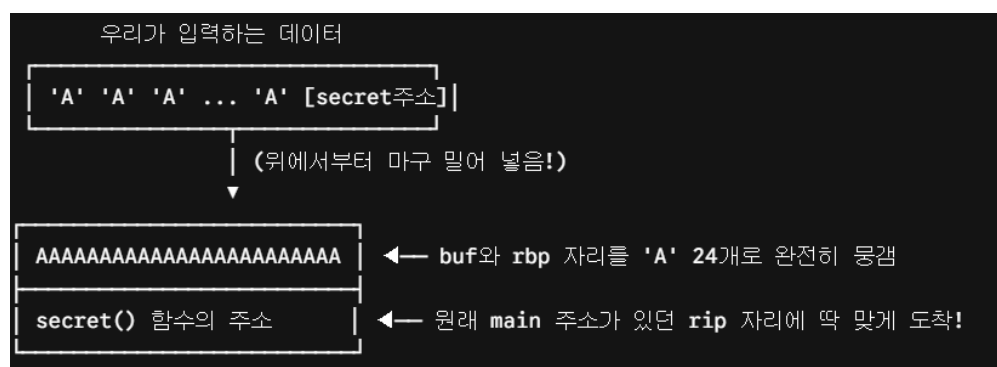
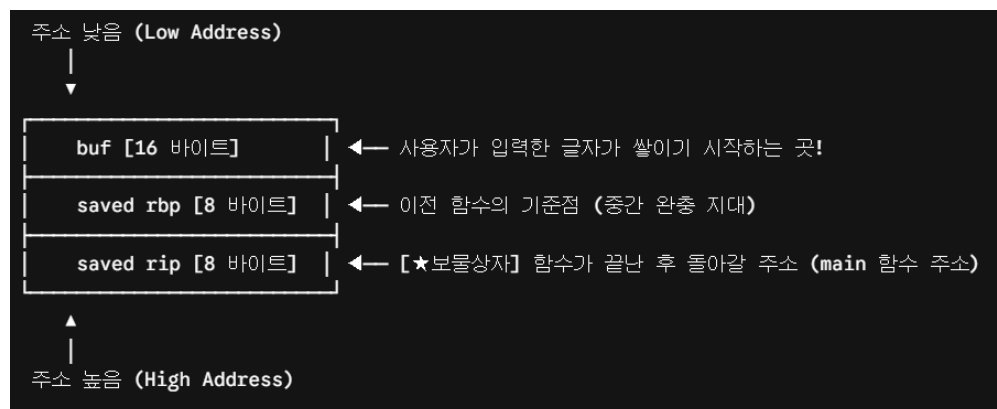
ebp (Extended Base Pointer): 옛날 32비트(4바이트) 컴퓨터 시절에 쓰던 레지스터

또한 `rip = 0x40117c` in `vuln (buff.c:13)`는 현재 프로그램이 `buff.c` 파일의 13번째 줄에 있는 `vuln` 함수 안의 `0x40117c`라는 코드 주소를 실행하는 중이라는 뜻이고 `saved rip`는 `main`으로 돌아갈 주소를 저장해놓는 레지스터임을 알 수 있다. 즉, `0x4011a2`는 `main`으로 돌아가서 실행될 주소인 것이다. 즉, `vuln` 함수가 끝나고 돌아갈 곳이기 때문에 여기에 시크릿 함수의 주소를 넣으면, `main` 함수가 아닌 시크릿 함수로 들어가게 될 것이라는 사실을 알 수 있다. 그리고 `Arglist at 0x7fffffffdde0`, `args: Locals at 0x7fffffffdde0`, `rbp at 0x7fffffffdde0` 이 3개가 모두 같은데, 마지막 두개가 같은 이유는 컴퓨터(GDB)가 `Locals`(지역 변수 구역)의 주소를 결정할 때는 **현재 `rbp` 레지스터가 가리키고 있는 주소 그 자체를 기준으로 잡기 때문이다**. 인자 또한 같은 주소인 이유는 `void vuln() {}`에서 `vuln()`의 괄호 안이 비어있기 때문에 전달받은 인자가 없고, 인자 리스트의 시작점(`Arglist`)을 스택 프레임의 기준점인 `rbp` 주소(`0x7fffffffdde0`)로 똑같이 설정한 것이기 때문이다. 만약 인자가 있는 함수였다면 이 주소는 `rbp`보다 더 아래쪽(높은 주소 방향)을 가리키게 된다.

설명이 길었는데, 따라서 `offset`을 계산하는 방식은 두가지 방식이 있다. 여기서 오프셋이란 기준점으로부터 얼마나 떨어져 있는지를 말한다.

1. `offset = buf 배열의 크기 + saved rbp의 크기` (여기서는 `16 + 8`)
2. `offset = saved rip의 주소 - buf의 시작 주소` (여기서는 `0x7fffffffdde8 - 0x7ffffffddd0`)

****16진수의 뺄셈은 16진수를 10진수로 고친다음 빼면된다.**



따라서 gets로 입력을 받을 때, 할당받은 buf보다 훨씬 큰 24바이트를 입력하게 되면, buf공간을 넘어 rbp공간까지 덮어쓰게 되고, 마지막에는 saved rip값까지 도달하게 된다는 것을 알 수 있다. 아까 p &secret 명령어를 통해 secret 함수의 주소가 0x401156인 것을 확인해 보았으므로 이제 익스플로잇을 진행해 보자.

python3 -c "print('A'*offset + secret_addr)" | ./vuln 의 형태로 넣으면 자꾸 broken pipe 에러가 떠서 인공지능의 도움을 받았다.

```
user@user-VirtualBox:~$ python3 -c "import sys; sys.stdout.buffer.write(b'A'*24 + b'\x56\x11\x40\x00\x00\x00\x00\x00') " | ./buff
secret function called!
Segmentation fault (core dumped)
```

이때 주소를 넣을 때는 다음과 같이 리틀 엔디안 방식으로 넣어야 한다. 컴퓨터는 데이터를 메모리에 저장할 때 바이트(byte) 단위로 나눠 저장하는데, 이 바이트 저장 순서에는 빅 엔디안과 리틀 엔디안이 있다. 리틀 엔디안(little endian)은 낮은 주소에 데이터의 낮은 바이트(LSB, Least Significant Bit)부터 저장하는 방식이다. 그림은 다음과 같다.

리틀 엔디안(Little Endian)



버퍼 오버플로우에서 리틀 엔디안 형식으로 익스플로잇을 진행하여야 하는 이유는 컴퓨터의 CPU 아키텍처(인텔 x64, RISC-V 등)가 메모리에 데이터를 저장할 때 리틀 엔디안 방식으로 저장하여서, 익스플로잇을 진행할 때에도 이에 맞게 넣어주어야 하기 때문이다. 여기서 메모리에 데이터를 저장할 때 리틀 엔디안 방식으로 저장하는 이유는 산술연산시 속도가 빨라지고, 주소가 0x0000012와 같이 있을 경우 빅엔디안 방식으로 하면 불필요한 연산을 수행하지만 리틀엔디안 방식으로 할 경우 뒤부터 바로 12 00 00과 같이 불필요한 연산을 수행할 필요가 없기 때문이다.

Challenge2. checksec으로 보호 기법 확인

우선 checksec은 리눅스 실행파일(ELF)에 적용된 보안 기능을 확인하는 도구이다.

예를 들어 checksec --file=vuln 를 실행하면

RELRO

Stack Canary

NX

PIE

...

등의 상태를 보여주는 도구이다.

가상환경에 checksec를 설치하고 checksec --file=./buff으로 실행해보면

```
user@user-VirtualBox:~$ checksec --file=./buff
RELRO           STACK CANARY      NX            PIE            RPATH          RUNPATH
Partial RELRO   No canary found    NX enabled    No PIE         No RPATH       No RUNPATH
37 Symbols      No                0             1              ./buff
```

이런 결과가 나오는 것을 알 수 있다. (-fno-stack-protector 옵션 사용)

반대로 gcc -g buff.c -o buff로 별도 옵션없이 컴파일하고 checksec를 수행하면

```
user@user-VirtualBox:~$ checksec --file=./buff
RELRO           STACK CANARY      NX            PIE            RPATH          RUNPATH
Full RELRO      Canary found      NX enabled    PIE enabled    No RPATH       No RUNPATH
40 Symbols      No                0             1              ./buff
```

다음과 같은 결과가 나오는 것을 확인할 수 있다.

각각의 변수에 대해 설명하자면 RELRO는 ReLocation Read-Only의 약어로 프로그램이 실행될 때 사용하는 외부 함수(예: printf, gets 등)의 메모리 주소가 적혀 있는 '함수 주소록(GOT)' 구역을 지킨다. 이전에는 이 주소록이 변경가능해서 해커가 gets 주소를 악성 코드 주소로 지우고 바꿔 쓰는 공격(GOT Overwrite)이 가능했다. RELRO는 이 주소록을 수정 불가능한 읽기 전용(Read-Only)으로 바뀌서 변조를 막는 기술이다.

Partial RELRO: 주소록의 일부만 잠그고 일부는 열어둔다. (취약함)

Full RELRO: 주소록 전체를 잠가서 보안을 유지한다. (안전함)

STACK CANARY는 함수가 끝난 후 원래 자리로 돌아갈 주소가 적힌 saved rip(리턴 주소)를 방어하는 것이다. (버퍼 오버플로우 방지)

buf 변수와 리턴 주소 사이에 컴퓨터만 아는 비밀번호(카나리 값)를 넣는다. 해커가 버퍼 오버플로우로 리턴 주소를 변조하려고 데이터를 밀어 넣으면 이 비밀번호가 먼저 까지게 된다. 컴퓨터는 함수가 끝나기 직전 카나리아가 죽었음(비밀번호가 바뀜)을 인지하고 프로그램을 강제로 터트려 공격을 차단한다. CANARY FOUND가 이 버퍼오버플로우를 방지할 수 있는 방법이 있다는 것이다.

NX (No-Execute)는 데이터를 저장하는 공간인 스택(Stack)이나 힙(Heap) 메모리 영역을 방어한다.

메모리 공간에 데이터만 저장하고, 코드로써 실행되지 않도록 실행 권한을 뺏어버리는 기술이다. 옛날 해커들은 buf에 직접 해킹용 코드(셸코드)를 적어두고 점프해서 실행하는 방식을 썼는데, NX가 켜져 있으면 buf에 적힌 코드를 실행하려는 순간 컴퓨터가 코드 실행 금지라며 프로그램을 차단한다. (Windows에서는 DEP)

PIE (Position Independent Executable)는 프로그램의 코드(텍스트) 영역 전체 주소를 방어한다. 프로그램이 메모리에서 실행될때마다 함수들의 주소를 매번 뒤섞어버린다. 이전 실습에서 secret 함수의 주소가 0x401156으로 고정되어 있어서 공격이 가능했는데, PIE가 켜져 있으면 쉼 때마다 0x55aa..., 0x7fff...로 주소가 계속 바뀌기 때문에 고정된 주소로는 공격이 불가능하게 된다. (메모리 전체를 섞는 ASLR 기술과 짝지어 작동)

RPATH / RUNPATH는 프로그램이 실행될 때 참고하는 외부 라이브러리(.so 파일)의 탐색 경로이다. 만약 이 경로에 시스템 기본 폴더가 아닌 . (현재 폴더) 같은 취약한 주소가 포함되어 있으면, 해커가 동일한 이름의 가짜 악성 라이브러리를 해당 폴더에 심어두는 '라이브러리 하이재킹' 공격에 노출될 수 있다. 프로그램이 실행되면서 취약한 주소를 먼저 찾아 실행시키게 되면 해커의 악성 코드가 프로그램 권한을 얻어 실행되는 취약점이 발생하는 것이다. checksec은 이 경로가 안전하게 비활성화되어 있는지(No RPATH / No RUNPATH) 감시한다.

Symbols는 프로그램 내부 변수나 함수의 이름이 남아있는지 확인하는 것이다. GDB에서 `print secret`이라고 쳤을 때 주소를 바로 찾을 수 있었던 이유는 프로그램 안에 `secret`이라는 이름 정보(Symbol)가 그대로 남아있었기 때문이다. 실제 배포용 프로그램을 만들 때는 해커들이 분석하기 어렵게 이 이름들을 지우는 '스트립(Strip)' 과정을 거치게 된다. No Symbols라고 쓰면 이름이 다 지워진 (보안이 강화된) 상태라는 뜻이다.

FORTIFY / Fortified / Fortifiable는 `strcpy`, `memcpy` 같이 길이를 검사하지 않는 위험한 소스코드 함수들을 방어한다. 컴파일러(GCC)가 코드를 검사하다가 위험한 함수를 발견하면, 이를 자동으로 길이를 검사하는 안전한 함수(`_strcpy_chk` 등)로 강제로 업그레이드(요새화, Fortify) 해주는 기술이다.

Fortifiable: 안전하게 바꿀 수 있는 잠재적인 위험 함수 개수

Fortified: 컴파일러가 실제로 안전하게 변환하는 데 성공한 함수의 개수

`-fno-stack-protector` 옵션 유무에 상관없이 Fortifiable이 1인 것으로 보아 안전하게 바꿀 만한 위험한 함수는 `gets()`인 것을 유추할 수 있고 Fortified가 0인 것으로 보아 컴파일할 때 이 옵션을 따로 활성화하지 않아서 실제로 안전하게 변환된 개수는 0이라는 것을 알 수 있다.